# Symbolic Range Analysis for Hybrid-Electric Aircraft

Derive, optimize, and deploy range equations for aircraft with combined fuel and battery energy sources where the classical Breguet range equation no longer applies.

**Workflow:** First-principles physics → symbolic integration → closed-form range equation → optimal energy split → sensitivity formulas → Monte Carlo range-margin analysis

Requires: Symbolic Math Toolbox™, Statistics and Machine Learning Toolbox™

Copyright 2026 The MathWorks, Inc.

## 1. Define the Hybrid-Electric Energy Model

A hybrid-electric aircraft draws thrust power from two sources:

- Fuel (fraction $\phi$ of total thrust power): burns fuel, so aircraft mass decreases over time
- Battery (fraction $1-\phi$): draws stored electrical energy, mass stays constant

Define the key variables symbolically and build the physics from first principles.

```
syms phi positive            % power split: fraction of thrust from fuel (0,1]
syms LD positive             % lift-to-drag ratio
syms SFC positive            % thrust-specific fuel consumption [1/s] (fuel engine)
syms V positive              % cruise velocity [m/s]
syms W0 positive             % initial gross weight [N]
syms xi positive             % fuel mass fraction W_fuel/W0
syms W_batt_frac positive    % battery mass fraction W_batt/W0
syms eta_e positive          % electric drivetrain efficiency
syms e_batt positive         % battery specific energy [J/kg]
syms g positive              % gravitational acceleration [m/s^2]
syms t                       % time variable for integration
```

In steady level cruise, lift equals weight and thrust equals drag:

$$L = W, \quad T = \frac{W}{L/D}$$

The fuel burn rate is proportional only to the fuel-powered fraction of thrust:

$$-\frac{dW}{dt} = \text{SFC} \cdot \phi \cdot T = \frac{\text{SFC} \cdot \phi \cdot W}{L/D}$$

The aircraft weight at time $t$ is:

$$W(t) = W_{\text{empty}} + W_{\text{batt}} + W_{\text{fuel}}(t)$$

where $W_{\text{batt}}$ is constant (batteries don't get lighter as they discharge).

```
syms W(t)
```

Weight dynamics (ODE):

```
weightODE = diff(W, t) == -SFC * phi * W / LD
```

```
weightODE(t) =
```

$$\frac{\partial}{\partial t} W(t) = -\frac{\text{SFC}\ \phi\ W(t)}{\text{LD}}$$

## 2. Derive the Hybrid Range Equation

Integrate the equations of motion to get range. The range increment is:

$$dR = V\ dt$$

From the weight ODE:

$$dt = \frac{-L/D}{\text{SFC} \cdot \phi} \cdot \frac{dW}{W}$$

Integrating from initial weight $W_0$ to final weight $W_f = W_0(1 - \xi)$:

```
syms W_var positive  % integration variable
integrand = V * LD / (SFC * phi * W_var);
W_final = W0 * (1 - xi);
R_hybrid = int(integrand, W_var, W_final, W0);
```

Hybrid range equation (symbolic):

```
R_hybrid = simplify(R_hybrid)
```

```
R_hybrid =
```

$$\begin{cases} -\dfrac{\mathrm{LD}\, V \log(1-\xi)}{\mathrm{SFC}\, \phi} & \text{if } \xi < 1 \end{cases}$$

## 3. Battery Constraint and Optimal Power Split

The mechanical power delivered by the electric motor at any instant is:

$$P_{\text{elec}} = (1-\phi) \cdot T \cdot V = \frac{(1-\phi) \cdot W \cdot V}{L/D}$$

The total mechanical energy drawn from the battery over the flight equals the time-integral of this power. Using the weight ODE to change variables from $dt$ to $dW$:

$$E_{\text{batt,consumed}} = \int (1-\phi) \frac{WV}{L/D} \cdot \frac{-L/D}{\text{SFC} \cdot \phi \cdot W} dW = \frac{(1-\phi)}{\phi} \cdot \frac{V \cdot W_{\text{fuel}}}{\text{SFC}}$$

This result follows because when we substitute the weight ODE, the $W$ and $L/D$ terms cancel, leaving a constant integrand. The integral simply evaluates to that constant times the total fuel weight $W_{\text{fuel}} = \xi W_0$.

The available battery energy (after drivetrain losses):

$$E_{\text{batt,available}} = \eta_e \cdot e_{\text{batt}} \cdot \frac{W_{\text{batt}}}{g}$$

```
E_batt_available = eta_e * e_batt * W_batt_frac * W0 / g;
E_batt_consumed = (1 - phi) / phi * V * xi * W0 / SFC;
```

Battery energy constraint (simultaneous depletion):

```
battConstraint = E_batt_available == E_batt_consumed
```

```
battConstraint =
```

$$\frac{W_0\, W_{\text{batt,frac}}\, e_{\text{batt}}\, \eta_e}{g} = -\frac{V\, W_0\, \xi\, (\phi - 1)}{\text{SFC}\, \phi}$$

### Solve for Optimal Power Split

Given the structural parameters (fuel fraction $\xi$, battery fraction), the constraint determines the power split $\phi$ that exactly exhausts both energy sources simultaneously, maximizing utilization.

```
phi_opt_sym = solve(battConstraint, phi);
```

Optimal power split (simultaneous depletion):

```
simplify(phi_opt_sym)
```

```
ans =
```

$$\frac{V\, g\, \xi}{V\, g\, \xi + \text{SFC}\, W_{\text{batt,frac}}\, e_{\text{batt}}\, \eta_e}$$

Introduce a dimensionless parameter $\beta$ that characterizes the battery-to-fuel energy ratio:

$$\beta = \frac{\eta_e \cdot e_{\text{batt}} \cdot f_{\text{batt}} \cdot \text{SFC}}{g \cdot V}$$

Then $\phi^* = \frac{\xi}{\xi + \beta}$ — the optimal split depends on how much battery energy is available relative to fuel energy, weighted by their respective conversion efficiencies.

## 4. Verify the Classical Breguet Limit

When $\phi = 1$ (all thrust from fuel, no battery), the hybrid range equation must reduce to the classical Breguet equation:

$$R_{\text{Breguet}} = \frac{V \cdot (L/D)}{\text{SFC}} \ln\left(\frac{1}{1 - \xi}\right)$$

Classical Breguet:

```
R_breguet = V * LD / SFC * log(1 / (1 - xi))
```

```
R_breguet =
```

$$\frac{\mathrm{LD}\, V \log\left(-\frac{1}{\xi - 1}\right)}{\mathrm{SFC}}$$

Hybrid range at phi=1:

```
R_at_phi1 = subs(R_hybrid, phi, 1);
simplify(R_at_phi1)
```

```
ans =
```

$$\begin{cases} -\frac{\mathrm{LD}\, V \log(1-\xi)}{\mathrm{SFC}} & \text{if } \xi < 1 \end{cases}$$

Difference (should be 0):

```
simplify(R_at_phi1 - R_breguet)
```

```
ans =
```

$$\begin{cases} 0 & \text{if } \xi < 1 \end{cases}$$

## 5. Derive the Optimized Range

The power split $\phi$ determines how the aircraft uses its two energy sources. If $\phi$ is too high (mostly fuel), battery energy is wasted. If $\phi$ is too low (mostly battery), the battery runs out before the fuel does.

The optimal $\phi$ simultaneously depletes both sources, maximizing range:

- $\phi < \phi^*$: battery runs out first (battery-limited)
- $\phi > \phi^*$: fuel runs out first (fuel-limited)
- $\phi = \phi^*$: both exhaust together

This assumes 100% exhaustion of both energy sources. In FAA certification workflows, a 45-minute cruise reserve must be reserved, which shifts $\xi_{\text{usable}} < \xi_{\text{total}}$.

### Range at Optimal Power Split

Substituting the optimal $\phi^*$ into the range equation gives the maximum achievable range:

```
R_opt_sym = subs(R_hybrid, phi, phi_opt_sym);
```

Maximum range (at optimal phi):

```
R_opt_sym = simplify(R_opt_sym)
```

```
R_opt_sym =
```

$$\begin{cases} -\dfrac{\mathrm{LD}\,\log(1-\xi)\;(V\,g\,\xi+\mathrm{SFC}\;W_{\mathrm{batt,frac}}\,e_{\mathrm{batt}}\,\eta_e)}{\mathrm{SFC}\;g\,\xi} & \text{if } \xi<1 \end{cases}$$

This decomposes as:

$$R^* = \left(\frac{V\cdot L/D}{\mathrm{SFC}} + \frac{L/D\cdot\eta_e\cdot e_{\mathrm{batt}}\cdot f_{\mathrm{batt}}}{g\cdot\xi}\right)\ln\!\left(\frac{1}{1-\xi}\right)$$

The first term is the classical Breguet prefactor. The second is the battery range bonus - an additive enhancement proportional to battery energy density and inversely proportional to fuel fraction.

Explicit form:

```
R_opt_explicit = (V*LD/SFC + LD*eta_e*e_batt*W_batt_frac/(g*xi)) * log(1/(1-xi))
```

```
R_opt_explicit =
```

$$\log\!\left(-\frac{1}{\xi-1}\right)\left(\frac{\mathrm{LD}\;V}{\mathrm{SFC}}+\frac{\mathrm{LD}\;W_{\mathrm{batt,frac}}\,e_{\mathrm{batt}}\,\eta_e}{g\,\xi}\right)$$

Verification (difference should be 0):

```
simplify(R_opt_explicit - R_opt_sym)
```

```
ans =
```

$$\begin{cases} 0 & \text{if } \xi<1 \end{cases}$$

## 6. Evaluate a Reference Aircraft

Use a ~50-seat hybrid-electric regional aircraft cruising at 25,000 ft and 280 kts. The analysis uses SI units internally; the conversion constants make the engineering assumptions explicit without introducing an extra toolbox dependency.

```
% Unit conversion constants
ft_to_m = 0.3048;
lbm_to_kg = 0.45359237;
kts_to_mps = 0.514444444444444;
N_to_lbf = 0.224808943099711;
nmi_to_m = 1852;
cruise_alt_ft = 25000;
cruise_alt_m = cruise_alt_ft * ft_to_m;
% Aircraft parameters
W0_val          = 55000 * lbm_to_kg * 9.81;             % gross weight [N]
LD_val          = 18;                                     % lift-to-drag ratio
SFC_val         = 0.6 / 3600;                             % SFC [1/s] (0.6 lb/lbf/hr)
V_val           = 280 * kts_to_mps;                       % cruise speed [m/s]
eta_e_val       = 0.85;                                   % electric drivetrain efficiency
e_batt_val      = 250 * 3600;                             % battery specific energy [J/kg]
W_batt_frac_val = 0.10;                                   % battery is 10% of gross weight
xi_val          = 0.20;                                   % fuel is 20% of gross weight
g_val           = 9.81;
% Derived quantities
beta_val = eta_e_val * e_batt_val * W_batt_frac_val * SFC_val / (g_val * V_val);
phi_opt_val = xi_val / (xi_val + beta_val);
fprintf(['=== Reference Design Point ===\n' ...
    'Cruise altitude:     %d ft (%.0f m)\n' ...
    'Gross weight:        %.0f N (%.0f lbf)\n' ...
    'L/D:                 %.1f\n' ...
    'SFC:                 %.4f 1/s (0.6 lb/lbf/hr)\n' ...
    'Cruise speed:        %.1f m/s (280 kts)\n' ...
    'Battery spec energy: %d Wh/kg\n' ...
    'Battery mass frac:   %.0f%%\n' ...
    'Fuel mass fraction:  %.0f%%\n' ...
    'Electric efficiency: %.0f%%\n' ...
    'Beta (energy ratio): %.4f\n' ...
    'Optimal phi:         %.4f\n'], ...
    cruise_alt_ft, cruise_alt_m, ...
```

```
    W0_val, W0_val * N_to_lbf, ...
    LD_val, ...
    SFC_val, ...
    V_val, ...
    e_batt_val/3600, ...
    W_batt_frac_val*100, ...
    xi_val*100, ...
    eta_e_val*100, ...
    beta_val, ...
    phi_opt_val)
```

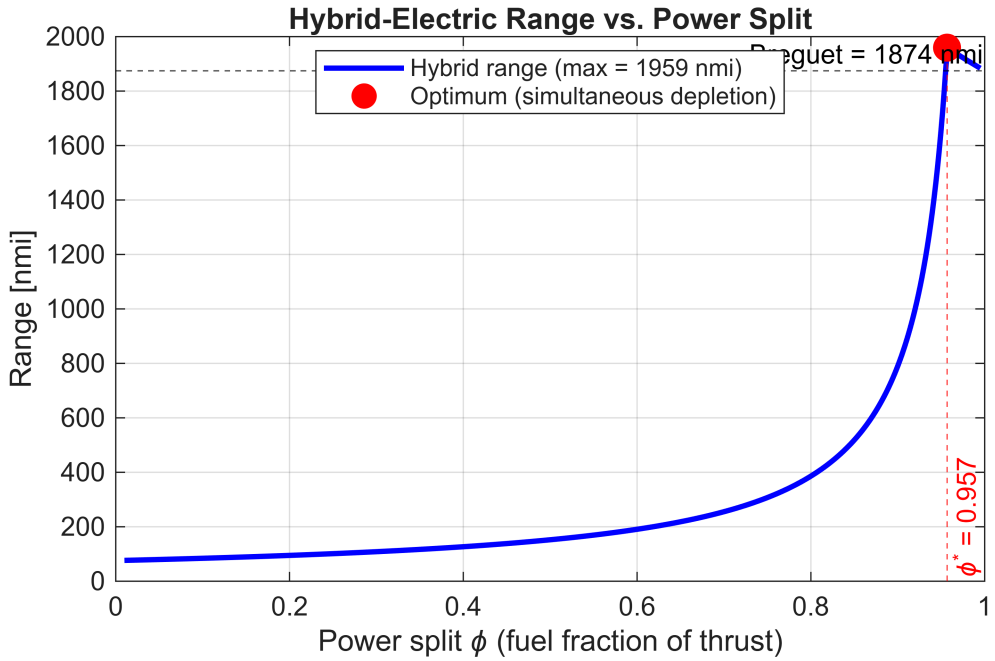
```
=== Reference Design Point ===
Cruise altitude:     25000 ft (7620 m)
Gross weight:        244736 N (55019 lbf)
L/D:                 18.0
SFC:                 0.0002 1/s (0.6 lb/lbf/hr)
Cruise speed:        144.0 m/s (280 kts)
Battery spec energy: 250 Wh/kg
Battery mass frac:   10%
Fuel mass fraction:  20%
Electric efficiency: 85%
Beta (energy ratio): 0.0090
Optimal phi:         0.9568
```

## Power-Split Regime Sweep

Evaluate range across a sweep of $\phi$ to visualize the battery-limited and fuel-limited regimes, with the optimum at their intersection.

```
phi_sweep = linspace(0.01, 0.995, 500);
nmi_per_m = 1 / nmi_to_m;
% In each regime, determine which resource is exhausted first:
% Battery-limited (phi < phi_opt): battery runs out, only partial fuel burned
%   xi_effective = beta * phi / (1-phi)  (from energy constraint)
% Fuel-limited (phi > phi_opt): all fuel burned, battery has leftover
%   xi_effective = xi_val
xi_batt_limited = beta_val * phi_sweep ./ (1 - phi_sweep);
xi_effective = min(xi_batt_limited, xi_val);
R_sweep_m = V_val * LD_val ./ (SFC_val .* phi_sweep) .* log(1 ./ (1 - xi_effective));
R_sweep_nm = R_sweep_m * nmi_per_m;
% Optimal range
```

```
R_opt_m = V_val * LD_val / (SFC_val * phi_opt_val) * log(1 / (1 - xi_val));
R_opt_nm = R_opt_m * nmi_per_m;
% Breguet comparison (phi=1, same fuel)
R_breguet_nm = V_val * LD_val / SFC_val * log(1 / (1 - xi_val)) * nmi_per_m;
plot(phi_sweep, R_sweep_nm, 'b-', 'LineWidth', 2)
hold on
plot(phi_opt_val, R_opt_nm, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r')
xline(phi_opt_val, '--r', sprintf('\\phi^* = %.3f', phi_opt_val), ...
    'LabelVerticalAlignment', 'bottom')
yline(R_breguet_nm, '--k', sprintf('Breguet = %.0f nmi', R_breguet_nm))
hold off
xlabel('Power split \phi (fuel fraction of thrust)')
ylabel('Range [nmi]')
title('Hybrid-Electric Range vs. Power Split')
legend(sprintf('Hybrid range (max = %.0f nmi)', R_opt_nm), ...
    'Optimum (simultaneous depletion)', 'Location', 'best')
grid on
```



```
fprintf(['\n=== Range Comparison ===\n' ...
    'Optimal hybrid range: %.0f nmi at phi = %.4f\n' ...
    'Breguet (fuel only):  %.0f nmi (phi = 1)\n' ...
    'Range improvement:    %.1f%%n'], ...
    R_opt_nm, phi_opt_val, ...
    R_breguet_nm, ...
    (R_opt_nm/R_breguet_nm - 1)*100)
```

```
=== Range Comparison ===
Optimal hybrid range: 1959 nmi at phi = 0.9568
Breguet (fuel only):  1874 nmi (phi = 1)
Range improvement:    4.5%n
```

## 7. Sensitivity Analysis

Which design parameters matter most for range? Symbolic differentiation gives exact formulas for how the optimized range responds to each parameter.

At the optimal power split, range is:

$$R^* = \frac{V \cdot L/D}{\mathrm{SFC}} \cdot \ln\left(\frac{1}{1-\xi}\right) + \frac{L/D \cdot \eta_e \cdot e_{\mathrm{batt}} \cdot f_{\mathrm{batt}}}{g \cdot \xi} \cdot \ln\left(\frac{1}{1-\xi}\right)$$

The first term is the fuel contribution (classical Breguet) and the second is the battery range bonus.

All parameters appear explicitly. The additive structure means fuel-side and battery-side sensitivities are cleanly separable.

### Analytic Partial Derivatives

Compute $\partial R^*/\partial p$ for each design parameter $p$:

```
design_params = [V, LD, SFC, eta_e, e_batt, W_batt_frac, xi];
param_names = {'V (cruise speed)', 'L/D', 'SFC', ...
    '\eta_e (electric eff.)', 'e_{batt} (battery energy)', ...
    'W_{batt}/W_0 (battery frac)', '\xi (fuel fraction)'};
n_params = length(design_params);
dR = sym(zeros(n_params, 1));
for i = 1:n_params
```

```
    dR(i) = diff(R_opt_explicit, design_params(i));
end
```

Symbolic range sensitivities dR*/d(param):

```
for i = 1:n_params
    fprintf('\n  dR/d(%s):\n%s', char(design_params(i)), evalc('disp(simplify(dR(i)))'))
end
```

```
  dR/d(V):
(LD*log(-1/(xi - 1)))/SFC


  dR/d(LD):
log(-1/(xi - 1))*(V/SFC + (W_batt_frac*e_batt*eta_e)/(g*xi))


  dR/d(SFC):
-(LD*V*log(-1/(xi - 1)))/SFC^2


  dR/d(eta_e):
(LD*W_batt_frac*e_batt*log(-1/(xi - 1)))/(g*xi)


  dR/d(e_batt):
(LD*W_batt_frac*eta_e*log(-1/(xi - 1)))/(g*xi)

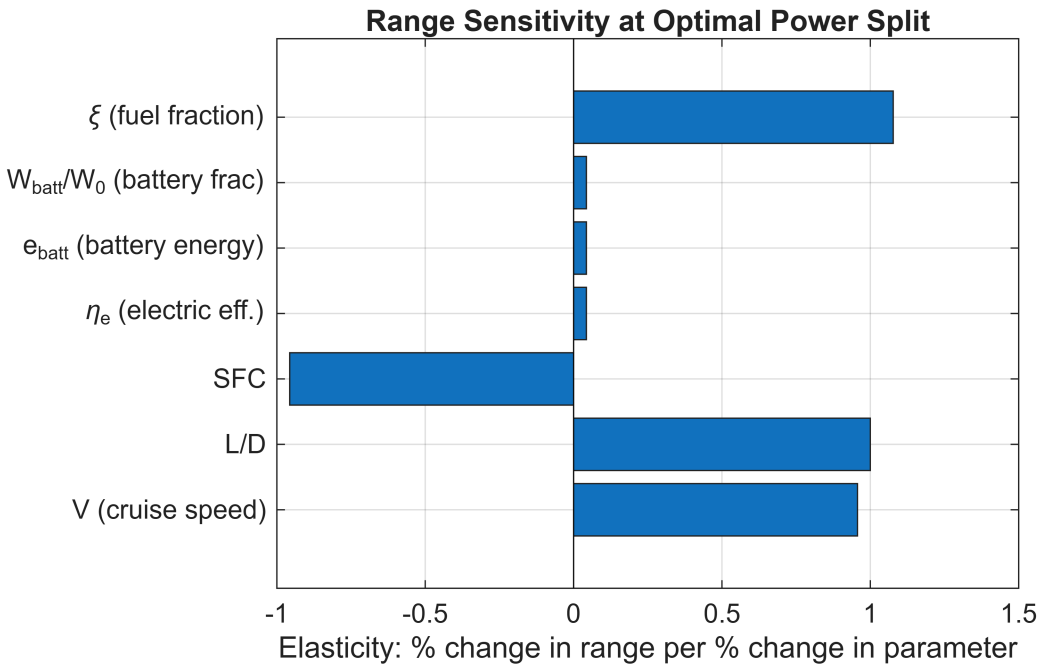

  dR/d(W_batt_frac):
(LD*e_batt*eta_e*log(-1/(xi - 1)))/(g*xi)


  dR/d(xi):
- ((LD*V)/SFC + (LD*W_batt_frac*e_batt*eta_e)/(g*xi))/(xi - 1) - (LD*W_batt_frac*e_batt*eta_e*log(-1/(xi - 1)))/(g*xi^2)
```

## Normalized Sensitivity (Elasticity)

To compare parameters on equal footing, normalize each sensitivity to an elasticity: multiply the partial derivative by the ratio of parameter value to range value. This gives the percentage change in range per percentage change in each parameter, allowing for direct comparison across parameters with different units.

```
elasticity = sym(zeros(n_params, 1));
for i = 1:n_params
    elasticity(i) = simplify(design_params(i) / R_opt_explicit * dR(i));
end
% Evaluate numerically at reference design point
cal_vars = [V, LD, SFC, eta_e, e_batt, W_batt_frac, xi, g];
cal_vals = [V_val, LD_val, SFC_val, eta_e_val, e_batt_val, W_batt_frac_val, xi_val, g_val];
elasticity_Vals = double(subs(elasticity, cal_vars, cal_vals));
barh(elasticity_Vals)
set(gca, 'YTickLabel', param_names, 'YTick', 1:n_params)
xlabel('Elasticity: % change in range per % change in parameter')
title('Range Sensitivity at Optimal Power Split')
grid on
xline(0, 'k')
```




```
fprintf('\n=== Range Elasticities at Design Point ===\n')
```

```
=== Range Elasticities at Design Point ===
```

```
for i = 1:n_params
    fprintf('  %-30s  %+.3f\n', param_names{i}, elasticity_Vals(i))
end
```

```
  V (cruise speed)               +0.957
  L/D                            +1.000
  SFC                            -0.957
  \eta_e (electric eff.)         +0.043
  e_{batt} (battery energy)      +0.043
  W_{batt}/W_0 (battery frac)    +0.043
  \xi (fuel fraction)            +1.077
```

```
fprintf(['\nFuel contribution to range: phi* = %.3f\n' ...
    'Battery contribution:       1-phi* = %.3f\n'], ...
    phi_opt_val, ...
    1-phi_opt_val)
```

```
Fuel contribution to range: phi* = 0.957
Battery contribution:       1-phi* = 0.043
```

## 8. Code Generation — Reusable Range Functions and C Expression

Convert the symbolic expressions into MATLAB® functions that downstream workflows (sizing tools, mission planners, Monte Carlo engines) can call directly. These functions accept vectorized inputs.

The generated range function evaluates the optimal range (assuming simultaneous depletion of fuel and battery):

```
range_vars = {V, LD, SFC, eta_e, e_batt, W_batt_frac, xi, g};
matlabFunction(R_opt_explicit, 'File', 'hybridRange', ...
    'Vars', range_vars, ...
    'Outputs', {'R_meters'});
% Sensitivity function: returns all partial derivatives
matlabFunction(dR(1), dR(2), dR(3), dR(4), dR(5), dR(6), dR(7), ...
    'File', 'hybridRangeSensitivity', ...
    'Vars', range_vars, ...
    'Outputs', {'dR_dV', 'dR_dLD', 'dR_dSFC', 'dR_deta_e', 'dR_de_batt', 'dR_dW_batt_frac', 'dR_dxi'});
% Optimal phi function
matlabFunction(phi_opt_sym, 'File', 'hybridOptimalPhi', ...
    'Vars', {V, SFC, eta_e, e_batt, W_batt_frac, xi, g}, ...
```

```
    'Outputs', {'phi_opt'});
```

For equation exchange with third-party modeling tools, `ccode` emits a compact C expression. In this application, that expression could be wrapped in a Modelica aircraft energy model.

```
cRangeExpression = ccode(R_opt_explicit)
```

```
cRangeExpression =
'  t0 = log(-1.0/(xi-1.0))*((LD*V)/SFC+(LD*W_batt_frac*e_batt*eta_e)/(g*xi));'
```

```
fprintf(['Generated: hybridRange.m\n' ...
    'Generated: hybridRangeSensitivity.m\n' ...
    'Generated: hybridOptimalPhi.m\n' ...
    '  MATLAB functions accept vectorized inputs for batch evaluation.\n' ...
    '  ccode(R_opt_explicit) emits a C expression for external model exchange.\n'])
```

```
Generated: hybridRange.m
Generated: hybridRangeSensitivity.m
Generated: hybridOptimalPhi.m
  MATLAB functions accept vectorized inputs for batch evaluation.
  ccode(R_opt_explicit) emits a C expression for external model exchange.
```

## 9. Monte Carlo — Range Margin Under Uncertainty

Assign probability distributions to uncertain parameters and evaluate the closed-form range function across thousands of scenarios. This shows how the symbolic result becomes a fast margin-analysis tool without an inner simulation loop.

### Define Parameter Distributions

```
N_scenarios = 50000;
rng(42)
% L/D: normal — manufacturing variability in wetted area, surface finish
pd_LD = makedist('Normal', 'mu', 18, 'sigma', 0.5);
pd_LD = truncate(pd_LD, 14, 22);
% SFC: lognormal — engine-to-engine variability, degradation over life
pd_SFC = makedist('Lognormal', 'mu', log(0.6/3600), 'sigma', 0.05);
% Battery specific energy: normal — cell-to-cell variability
pd_ebatt = makedist('Normal', 'mu', 250*3600, 'sigma', 15*3600);
```

```
pd_ebatt = truncate(pd_ebatt, 150*3600, 350*3600);
% Electric drivetrain efficiency: beta distribution on [0.75, 0.95]
pd_eta_raw = makedist('Beta', 'a', 8, 'b', 2);
% Cruise speed: narrow normal — pilot/ATC variability
pd_V = makedist('Normal', 'mu', 280 * kts_to_mps, 'sigma', 5 * kts_to_mps);
pd_V = truncate(pd_V, 260 * kts_to_mps, 300 * kts_to_mps);
% Draw samples
LD_draws    = random(pd_LD, N_scenarios, 1);
SFC_draws   = random(pd_SFC, N_scenarios, 1);
ebatt_draws = random(pd_ebatt, N_scenarios, 1);
eta_draws   = 0.75 + 0.20 * random(pd_eta_raw, N_scenarios, 1);
V_draws     = random(pd_V, N_scenarios, 1);
fprintf(['=== Monte Carlo Setup ===\n' ...
    'Scenarios:        %d\n' ...
    'Uncertain params: L/D, SFC, e_batt, eta_e, V\n' ...
    'Fixed params:     xi, W_batt/W0, g\n'], ...
    N_scenarios)
```

```
=== Monte Carlo Setup ===
Scenarios:        50000
Uncertain params: L/D, SFC, e_batt, eta_e, V
Fixed params:     xi, W_batt/W0, g
```

## Evaluate Range Distribution

One vectorized function call evaluates the closed-form range across all scenarios - the same `hybridRange` function generated from the symbolic derivation.

```
R_scenarios = hybridRange(V_draws, LD_draws, SFC_draws, ...
    eta_draws, ebatt_draws, W_batt_frac_val, xi_val, g_val);
R_scenarios_nm = R_scenarios * nmi_per_m;
% Compute statistics
R_mean = mean(R_scenarios_nm);
R_std  = std(R_scenarios_nm);
R_05   = quantile(R_scenarios_nm, 0.05);
R_01   = quantile(R_scenarios_nm, 0.01);
R_required_nm = 1800;
P_meet_empirical = mean(R_scenarios_nm >= R_required_nm);
```

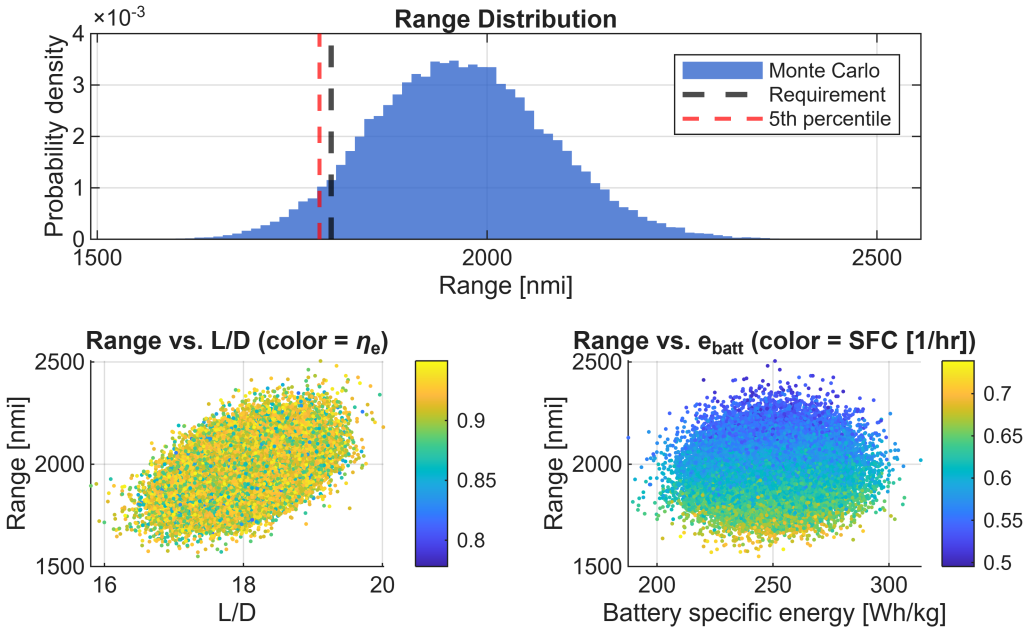
```
fprintf(['\n=== Range Distribution ===\n' ...
    'Mean range:          %.0f nmi\n' ...
    'Std deviation:       %.0f nmi\n' ...
    '5th percentile:      %.0f nmi\n' ...
    '1st percentile:      %.0f nmi\n' ...
    'P(Range >= %d nmi):  %.4f\n'], ...
    R_mean, ...
    R_std, ...
    R_05, ...
    R_01, ...
    R_required_nm, P_meet_empirical)
```

```
=== Range Distribution ===
Mean range:          1967 nmi
Std deviation:       114 nmi
5th percentile:      1785 nmi
1st percentile:      1713 nmi
P(Range >= 1800 nmi):  0.9346
```

## Visualize Results

```
tiledlayout(2,2)
nexttile([1 2])
histogram(R_scenarios_nm, 80, 'FaceColor', [0.2 0.4 0.8], 'EdgeColor', 'none', ...
    'FaceAlpha', 0.8, 'Normalization', 'pdf', 'DisplayName', 'Monte Carlo')
hold on
xline(R_required_nm, 'k--', 'LineWidth', 2, 'DisplayName', 'Requirement')
xline(R_05, 'r--', 'LineWidth', 1.5, 'DisplayName', '5th percentile')
hold off
xlabel('Range [nmi]')
ylabel('Probability density')
title('Range Distribution')
legend('Location', 'northeast')
grid on
nexttile
scatter(LD_draws, R_scenarios_nm, 2, eta_draws, 'filled')
colorbar
xlabel('L/D')
```

```
ylabel('Range [nmi]')
title('Range vs. L/D (color = \eta_e)')
grid on
nexttile
scatter(ebatt_draws/3600, R_scenarios_nm, 2, SFC_draws*3600, 'filled')
colorbar
xlabel('Battery specific energy [Wh/kg]')
ylabel('Range [nmi]')
title('Range vs. e_{batt} (color = SFC [1/hr])')
grid on
```




## 10. Variance Decomposition via Analytic Sensitivities

The generated sensitivity function gives exact partial derivatives at every scenario point. This enables first-order variance decomposition to identify which uncertainties dominate range variability.

```
clf
[dR_dV_vals, dR_dLD_vals, dR_dSFC_vals, dR_deta_vals, dR_debatt_vals, ~, ~] = ...
```

```
    hybridRangeSensitivity(V_draws, LD_draws, SFC_draws, ...
    eta_draws, ebatt_draws, W_batt_frac_val, xi_val, g_val);
% First-order variance decomposition: Var(R) ~ sum (dR/dp_i)^2 * Var(p_i)
mean_dR = [mean(dR_dV_vals), mean(dR_dLD_vals), mean(dR_dSFC_vals), ...
           mean(dR_deta_vals), mean(dR_debatt_vals)];
param_vars = [var(V_draws), var(LD_draws), var(SFC_draws), ...
              var(eta_draws), var(ebatt_draws)];
var_contributions = mean_dR.^2 .* param_vars;
var_contributions = var_contributions / sum(var_contributions) * 100;
bar(var_contributions)
set(gca, 'XTickLabel', {'V', 'L/D', 'SFC', '\eta_e', 'e_{batt}'})
ylabel('% of range variance')
title('Which Uncertainties Drive Range Variability?')
grid on
```

